1. Variables and Scope

Difference between var, let, and const

Explanation: `var` is function-scoped and can be re-declared. `let` and `const` are block-scoped (live only inside `{}`). `let` can be updated, but `const` cannot be updated or re-declared.

```
let name = "Amrit";
name = "Raj"; // Allowed

const stack = "MERN";
stack = "MEAN"; // Error: Assignment to constant variable.

if(true) {
    var a = 10;
    let b = 20;
}
console.log(a);
console.log(b);
```

```
Output:
10
ReferenceError: b is not defined
```

Why this output comes: `var` leaks out of the `if` block, but `let` stays trapped inside the `{}` block.

> 💡 **Interview Tip:** Always use `const` by default. Use `let` only if the value will change. Never use `var` in modern JavaScript.

Hoisting & Temporal Dead Zone (TDZ)

Explanation: Hoisting is JavaScript's default behavior of moving declarations to the top before execution. `var` is hoisted with undefined. `let` and `const` are hoisted but go into a **TDZ** (Temporal Dead Zone), meaning you cannot access them until their exact line of code is run.

```
console.log(x);
console.log(y);

var x = 5;
let y = 10;
```

> **Output:**
> undefined
> ReferenceError: Cannot access 'y' before initialization

Why this output comes: `x` (var) is hoisted and initialized as `undefined`. `y` (let) is hoisted but stays in the TDZ until line 4.

> 💡 **Interview Tip:** TDZ is simply the time between entering a scope and the variable being declared. It prevents you from using variables too early.

2. Functions

Function Declaration vs Function Expression

Explanation: A **Declaration** is a standard function creation and gets fully hoisted (you can call it before you write it). An **Expression** stores a function in a variable and is NOT hoisted.

```
sayHi(); // Works
sayBye(); // Error

function sayHi() {
    console.log("Hi");
}

const sayBye = function() {
    console.log("Bye");
};
```

> **Output:**
> "Hi"
> ReferenceError: Cannot access 'sayBye' before initialization

Why this output comes: Function declarations are loaded into memory completely before execution. Expressions act like normal variables (TDZ applies here).

> 💡 **Interview Tip:** Use declarations for global helper functions, and expressions (like arrow functions) for passing callbacks.

Arrow Function vs Normal Function

Explanation: Arrow functions are a shorter syntax introduced in ES6. They do not have their own `this` (they inherit it), and they do not have the `arguments` object. Normal functions have both.

```
const addNormal = function(a, b) {
    return a + b;
};

// Shorter Arrow Function
const addArrow = (a, b) => a + b;

console.log(addNormal(2, 3));
console.log(addArrow(2, 3));
```

```
Output:
5
5
```

Why this output comes: Both do the same thing, but the arrow function has an implicit (automatic) return when there are no curly braces `{}`.

> 💡 **Interview Tip:** If the interviewer asks the main difference, always say "Arrow functions do not bind their own 'this' keyword."

3. Closures

What is a Closure? (With Counter Example)

Explanation: A closure is when an inner function "remembers" the variables of its outer function, even after the outer function has finished running. It is like a backpack of memories.

```
function createCounter() {
    let count = 0; // Outer variable
    return function() {
        count++; // Inner function remembers 'count'
        return count;
    };
}

const counter = createCounter(); // createCounter finishes running here
console.log(counter());
console.log(counter());
```

Output:
1
2

Why this output comes: Even though `createCounter` finished, the returned inner function still has access to the `count` variable. This private memory is the closure.

> 💡 **Interview Tip:** Closures are commonly used in React (MERN stack) for maintaining state in hooks like `useState`.

4. The 'this' Keyword

'this' in Normal vs Object vs Arrow Functions

Explanation: The value of `this` changes based on HOW a function is called.
- **Normal function:** Refers to the global object (Window).
- **Object method:** Refers to the object calling the method.
- **Arrow function:** Takes `this` from its surrounding environment (lexical scope).

```
const user = {
    name: "Developer",
    normalMethod: function() {
        console.log(this.name);
    },
    arrowMethod: () => {
        console.log(this.name);
    }
};

user.normalMethod();
user.arrowMethod();
```

```
Output:
"Developer"
undefined
```

Why this output comes: For `normalMethod`, `this` is the `user` object. For `arrowMethod`, `this` is the global Window object (which doesn't have a `name` property).

> 💡 **Interview Tip:** Never use arrow functions as object methods if you need to access other properties of that object using `this`.

5. Call, Apply, and Bind

Difference and Examples

Explanation: These methods allow you to manually set the value of `this` for a function.
- **Call:** Passes arguments one by one (comma-separated).
- **Apply:** Passes arguments as an **Array** (A for Apply, A for Array).
- **Bind:** Does not execute immediately. Returns a new function to be called later.

```
const platform = { name: "Career Guidance App" };

function displayInfo(tech, year) {
    console.log(this.name + " in " + tech + " (" + year + ")");
}

// Call (comma)
displayInfo.call(platform, "React", 2025);

// Apply (array)
displayInfo.apply(platform, ["Node.js", 2025]);

// Bind (returns new function)
const bindedFunc = displayInfo.bind(platform, "MongoDB", 2025);
bindedFunc();
```

```
Output:
Career Guidance App in React (2025)
Career Guidance App in Node.js (2025)
Career Guidance App in MongoDB (2025)
```

Why this output comes: We force the `this` inside `displayInfo` to point to the `platform` object using call/apply/bind.

> 💡 **Interview Tip:** Bind is heavily used in event listeners or when passing methods as callbacks to ensure `this` isn't lost.

6. Objects & Arrays

Shallow Copy vs Deep Copy

Explanation: A **Shallow Copy** copies the top-level properties. If there are nested objects, they still share the same reference. A **Deep Copy** creates a completely independent copy of all nested data.

```
const original = { name: "Amrit", skills: ["JS", "React"] };

// Shallow Copy using Spread Operator
const shallow = { ...original };
shallow.skills.push("Node");

console.log(original.skills);

// Deep Copy using JSON methods
const deep = JSON.parse(JSON.stringify(original));
deep.skills.push("Python");

console.log(original.skills);
```

Output:
```
["JS", "React", "Node"]
["JS", "React", "Node"]
```

Why this output comes: Modifying the nested `skills` array in the shallow copy affected the original because they share the same memory reference. The deep copy did not affect the original.

> 💡 **Interview Tip:** `structuredClone()` is the modern built-in way to do deep copying in JS instead of the JSON trick.

Map, Filter, Reduce & forEach

Explanation:
- **Map:** Transforms elements and returns a NEW array of the same length.
- **Filter:** Returns a NEW array containing only elements that pass a condition.
- **Reduce:** Takes all elements and reduces them into a single value (like sum).
- **forEach:** Loops over the array but returns undefined. It does not create a new array.

```
const nums = [1, 2, 3, 4];

// Map vs forEach
const mapped = nums.map(n => n * 2);
const forEached = nums.forEach(n => n * 2);

console.log(mapped);
console.log(forEached);

// Filter
const evens = nums.filter(n => n % 2 === 0);
console.log(evens);

// Reduce (accumulator, current_value)
const sum = nums.reduce((acc, curr) => acc + curr, 0);
console.log(sum);
```

Output:
```
[2, 4, 6, 8]
undefined
[2, 4]
10
```

Why this output comes: `map` creates a new array, `forEach` just executes logic and returns nothing. `filter` removes odds, `reduce` adds 0+1+2+3+4.

> 💡 **Interview Tip:** If you need to chain methods (like array.filter().map()), you cannot use forEach because it returns undefined.

7. Asynchronous JavaScript

Callbacks & Callback Hell

Explanation: A **Callback** is a function passed as an argument to another function to be executed later (usually after a task finishes). **Callback Hell** happens when callbacks are nested inside other callbacks heavily, making code look like a pyramid and hard to read.

```
getUser(function(user) {
    getOrders(user.id, function(orders) {
        getPayment(orders[0], function(payment) {
            console.log("Payment completed!");
        });
    });
});
```

> **Output:**
> "Payment completed!" (after some time)

Why this output comes: JavaScript doesn't wait for one task to finish to start the next. Callbacks tell it what to do once the async task (like a DB fetch) is finally done.

> 💡 **Interview Tip:** To fix callback hell, JavaScript introduced Promises and later async/await.

Promises & States

Explanation: A Promise is an object representing the eventual success or failure of an asynchronous operation. It has 3 states:
1. **Pending:** Task is ongoing.
2. **Fulfilled:** Task finished successfully (handled by `.then()`).
3. **Rejected:** Task failed (handled by `.catch()`).
`.finally()` runs regardless of success or failure.

```
const checkData = new Promise((resolve, reject) => {
    let success = true;
    if(success) resolve("Data Loaded!");
    else reject("Error Loading");
});

checkData
    .then(result => console.log(result))
    .catch(error => console.log(error))
    .finally(() => console.log("Task Finished"));
```

```
Output:
Data Loaded!
Task Finished
```

Why this output comes: The promise calls `resolve`, pushing the string to `.then()`. Finally block always runs at the end.

> 💡 **Interview Tip:** Promises help write cleaner asynchronous code compared to messy nested callbacks.

async / await & What if 'await' is missed?

Explanation: `async/await` is syntactic sugar on top of Promises. It makes async code look synchronous and easier to read. The `await` keyword pauses execution until the Promise resolves.

```
async function fetchData() {
    return "API Data"; // Returns a Promise!
}

async function main() {
    const dataWithoutAwait = fetchData();
    console.log(dataWithoutAwait);

    const dataWithAwait = await fetchData();
    console.log(dataWithAwait);
}
main();
```

```
Output:
Promise { <pending> }
"API Data"
```

Why this output comes: If you forget `await`, JS does not wait for the result and simply logs the Promise object itself. Using `await` unwraps the resolved value.

> 💡 **Interview Tip:** An `async` function ALWAYS returns a Promise, even if you return a simple string inside it.

8. Fetch API

Promises vs async/await & response.json()

Explanation: Fetch is used to make network requests. The response from `fetch` is a readable stream, so we MUST call `response.json()` to parse the body into a usable JavaScript object.

```
// Using Promises
fetch('https://jsonplaceholder.typicode.com/users/1')
    .then(response => response.json()) // Parse JSON
    .then(data => console.log("Promises:", data.name));

// Same using async/await
async function getUser() {
    const response = await fetch('https://jsonplaceholder.typicode.com/users/1');
    const data = await response.json(); // Parse JSON
    console.log("Async/Await:", data.name);
}
getUser();
```

```
Output:
Promises: Leanne Graham
Async/Await: Leanne Graham
```

Why this output comes: Both do the exact same network call. `response.json()` itself returns a Promise, which is why we need `.then()` or `await` for it as well.

💡 **Interview Tip:** Always wrap async/await fetch calls in `try...catch` blocks to handle network errors properly.

9. Event Loop & Macrotask vs Microtask

Event Loop Architecture & Predict the Output

Explanation: JS is single-threaded. The **Call Stack** runs sync code. Async operations go to **Web APIs**. When done, callbacks move to Queues. The **Event Loop** checks: if Call Stack is empty, it pushes tasks from queues to the stack.
- **Microtasks (Promises, queueMicrotask):** High priority.
- **Macrotasks (setTimeout, setInterval):** Low priority.

```
console.log("Start");

setTimeout(() => {
    console.log("Macrotask (setTimeout)");
}, 0);

Promise.resolve().then(() => {
    console.log("Microtask (Promise)");
});

console.log("End");
```

```
Output:
Start
End
Microtask (Promise)
Macrotask (setTimeout)
```

Why this output comes: 1) Sync code ("Start", "End") runs first. 2) Microtasks (Promises) are checked next and executed. 3) Macrotasks (setTimeout) are executed last.

> 💡 **Interview Tip:** Microtasks ALWAYS jump the line ahead of Macrotasks. Remember: Call Stack -> Microtask Queue -> Macrotask Queue.

10. Event Propagation

Bubbling, Capturing, stopPropagation & preventDefault

Explanation: When you click a button inside a div, who gets the click first?
- **Capturing:** Event travels DOWN from Window -> div -> button.
- **Bubbling (Default):** Event travels UP from button -> div -> Window.
`stopPropagation()` stops this travel. `preventDefault()` stops the default browser action (like a form reloading the page).

```
document.querySelector('div').addEventListener('click', () => {
    console.log('Div clicked');
});

document.querySelector('button').addEventListener('click', (e) => {
    e.stopPropagation(); // Stops the bubble from reaching the div
    e.preventDefault();  // Stops form submission if it's a submit button
    console.log('Button clicked');
});
```

```
Output (when button clicked):
Button clicked
```

Why this output comes: Normally, clicking the button would trigger the button click, then bubble up and trigger the div click. `stopPropagation` kills the bubble.

> 💡 **Interview Tip:** To use capturing instead of bubbling, pass `{ capture: true }` as the 3rd argument to addEventListener.

11. Debouncing

What is Debouncing? (Search Input Example)

Explanation: Debouncing ensures that a time-consuming function does not fire so often. It delays the execution until a certain amount of time has passed since the user stopped typing or triggering the event.

```
function debounce(func, delay) {
    let timer;
    return function(...args) {
        clearTimeout(timer); // Reset the clock
        timer = setTimeout(() => {
            func.apply(this, args);
        }, delay);
    };
}

const searchAPI = (query) => console.log("Fetching API for:", query);
const debouncedSearch = debounce(searchAPI, 500);

// User types "A", "m", "r", "i", "t" quickly
debouncedSearch("A");
debouncedSearch("Am");
debouncedSearch("Amr");
debouncedSearch("Amri");
debouncedSearch("Amrit"); // Only this one will execute after 500ms pause
```

> **Output (after 500ms):**
> Fetching API for: Amrit

Why this output comes: Every quick keystroke resets the 500ms timer. The API is only called once the user stops typing for half a second. This saves server costs.

> 💡 **Interview Tip:** Debouncing is for a "pause" in typing. Throttling is for continuous actions like "scrolling", running a function once every X milliseconds.

12. Object-Oriented Programming (OOP)

Classes, Constructors, and Inheritance

Explanation: OOP involves organizing code into objects.
- **Encapsulation:** Hiding data (using `#` private fields).
- **Abstraction:** Hiding complex logic from the user.
- **Inheritance:** Creating a child class from a parent class using `extends`.
- **Polymorphism:** Method overriding (Child redefines Parent's method).

```
class User {
    constructor(name) {
        this.name = name;
    }
    getRole() {
        return "General User";
    }
}

class Admin extends User {
    constructor(name, power) {
        super(name); // Calls Parent constructor
        this.power = power;
    }
    // Polymorphism: Overriding getRole
    getRole() {
        return "Super Admin Level: " + this.power;
    }
}

const u1 = new Admin("Amrit", 5);
console.log(u1.name);
console.log(u1.getRole());
```

```
Output:
Amrit
Super Admin Level: 5
```

Why this output comes: `super()` initializes the parent properties. `getRole()` in Admin overrides the one in User.

> 💡 **Interview Tip:** Behind the scenes, JS Classes are just syntactic sugar over **Prototypes**. A Prototype is a fallback object that JS checks if a property isn't found on the main object.

13. Equality Operators

Difference between == and ===

Explanation:
== (Loose Equality): Compares only the values. It performs **Type Coercion** (converts string to number before checking).
=== (Strict Equality): Compares both values AND data types. No coercion.

```
console.log(5 == "5");
console.log(5 === "5");
console.log(false == 0);
console.log(false === 0);
console.log(null == undefined);
console.log(null === undefined);
```

```
Output:
true
false
true
false
true
false
```

Why this output comes: "5" string is converted to number 5 for `==`. For `===`, Number vs String is instantly false. Same for boolean vs number.

💡 **Interview Tip:** Always use `===` in production to prevent weird type conversion bugs.

⏱️ 30-Minute Interview Revision Cheat Sheet

Memorize these 15 key points before walking into your interview.

Question	Quick One-Line Answer
1. What is Hoisting?	JS moving variable and function declarations to the top of their scope before execution.
2. What is a Closure?	An inner function that remembers the variables of its outer function even after it returns.
3. var vs let vs const?	

	var is function-scoped. let/const are block-scoped. const cannot be reassigned.
4. What is TDZ?	The zone where let/const exist but cannot be accessed until their declaration line is executed.
5. Arrow functions vs Normal?	Arrow functions don't have their own `this` (they inherit it) and don't have `arguments`.
6. Call vs Apply vs Bind?	Call (comma args), Apply (array args) invoke immediately. Bind returns a new bound function.
7. Map vs forEach?	Map returns a new modified array. forEach returns undefined.
8. Shallow vs Deep Copy?	Shallow copies top-level references. Deep creates completely disconnected new memory allocations.
9. Promise states?	Pending (working), Fulfilled (success), Rejected (failure).
10. Macrotask vs Microtask?	Microtasks (Promises) have higher priority and run before Macrotasks (setTimeout).
11. Event Bubbling?	Event starts at the target element and flows UP to the window.
12. Event Capturing?	Event starts at the window and flows DOWN to the target element.
13. Debouncing?	Delaying function execution until a specific time has passed since the last trigger.
14. == vs === ?	== checks value (does type coercion). === checks value AND data type strictly.
15. What is the Event Loop?	The mechanism that pushes tasks from the callback queues to the call stack when it's empty.

🛑 Common Student Mistakes:

- Forgetting that `typeof null` returns `"object"` (This is a famous legacy JS bug!).
- Thinking that `const array = []` means you can't push items. You CAN push items, you just can't reassign `array = [1,2]`.
- Confusing `slice()` (returns shallow copy, doesn't modify original) with `splice()` (modifies original array).

⚡ Quick Output Question to Test Yourself:

```
console.log(1);
setTimeout(() => console.log(2), 0);
```

```
Promise.resolve().then(() => console.log(3));
console.log(4);
```

Answer: 1, 4, 3, 2. (Sync runs first -> 1 & 4. Microtasks next -> 3. Macrotasks last -> 2).